Avnik Local Development Setup

Running **avnik-admin** & **avnik-meeting** on Windows using WSL2, Docker & Nginx

What this guide covers

- Installing WSL2 + Ubuntu on Windows
- Essential Linux commands (from zero)
- Installing Docker Engine inside WSL
- Installing & configuring Nginx
- Creating the SSL certificate
- Adding the local domain to the hosts file
- Building & running both projects with Docker
- Verifying, troubleshooting & daily use

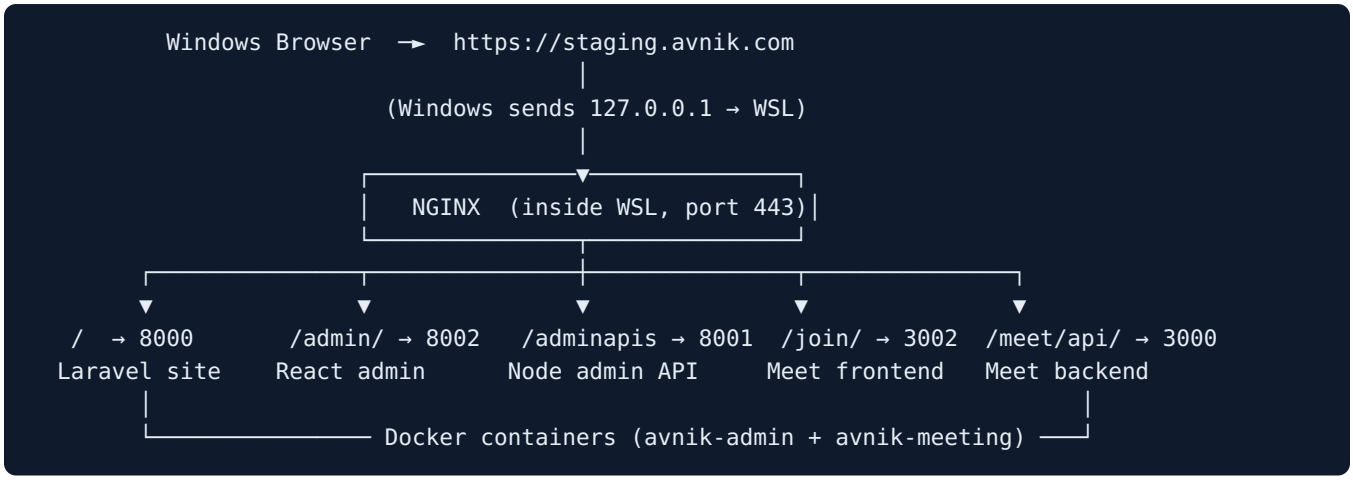
Target: Windows 10 (2004+) / Windows 11 · No source-code changes required
Local domain: **https://staging.avnik.com**

Table of Contents

1. Big picture: what you are building
2. Part 1 — Install WSL2 & Ubuntu on Windows
3. Part 2 — Linux survival kit (commands you must know)
4. Part 3 — Enable systemd inside WSL
5. Part 4 — Install Docker Engine inside WSL
6. Part 5 — Install Nginx inside WSL
7. Part 6 — Create the SSL certificate
8. Part 7 — Add the Nginx site configuration
9. Part 8 — Point the domain (Windows hosts file)
10. Part 9 — Get the project files into WSL
11. Part 10 — Create the `.env` files
12. Part 11 — Build & run avnik-admin
13. Part 12 — Build & run avnik-meeting
14. Part 13 — Start Nginx & open the apps
15. Part 14 — Daily start / stop routine
16. Part 15 — Troubleshooting
17. Appendix — Command cheat sheet

1. Big picture: what you are building

Both projects are made of several **Docker containers**. A single **Nginx** web server sits in front of everything and routes each URL path to the correct container. Everything runs **inside WSL** (a real Linux environment on your Windows PC), so it behaves exactly like the Linux server — **no code changes needed**.



Port map — you do NOT change these, they must match Nginx

avnik-admin project:

Service	Container name	Host port	Nginx path
Laravel backend (main website)	avnik-laravel-backend-prod	8000	`/`
Node admin API	avnik-admin-backend-prod	8001	`/adminapis`
React admin dashboard	avnik-admin-frontend-prod	8002	`/admin/`
phpMyAdmin (DB web UI)	avnik-phpmyadmin-prod	8080	`/myadmin/`
MySQL database	avnik-mysql-prod	internal only	—

avnik-meeting project:

Service	Container name	Host port	Nginx path
Meet backend (NestJS)	meet-backend-prod	3000	`/meet/api/` , `/socket.io/`
Meet media server	meet-mediaserver-prod	3001 (host network)	`/mediasoup-socket.io/`
Meet frontend	meet-frontend-prod	3002	`/join/`
Meet recorder	meet-recorder-prod	4000	internal
Postgres / Redis	meet-postgres-prod / meet-redis-prod	internal only	—

> **Very important — use Docker Engine inside WSL, NOT Docker Desktop.**
> The `meet-mediaserver` container uses `network_mode: host`, which only works correctly with the native Docker Engine running inside the WSL Linux distro. Docker Desktop's host-networking behaves differently and will break the media server. This guide installs Docker *inside* Ubuntu so it matches the Linux server 1-to-1.

Part 1 — Install WSL2 & Ubuntu on Windows

WSL (Windows Subsystem for Linux) runs a full Ubuntu Linux inside Windows. This is where everything will run.

Step 1.1 — Open PowerShell as Administrator

1. Click the Windows **Start** button.
2. Type `PowerShell`.
3. Right-click **Windows PowerShell → Run as administrator**.
4. Click **Yes** on the security prompt.

Step 1.2 — Install WSL + Ubuntu with one command

```
wsl --install -d Ubuntu-22.04
```

This enables WSL2 and downloads Ubuntu 22.04. When it finishes, **restart the computer** if it asks you to.

> If `wsl --install` says it is already installed, just run `wsl --install -d Ubuntu-22.04` to add the Ubuntu distro. To see installed distros: `wsl --list --verbose` (the `VERSION` column must show **2**).

Step 1.3 — Create your Linux user

After restart, Ubuntu opens a black terminal window and asks you to create a user:

- **Enter new UNIX username:** type a simple lowercase name, e.g. `dev`
- **New password:** type a password (the screen shows nothing while typing — that is normal), press `Enter`, then type it again.

Remember this password — you will use it with `sudo` (the command that runs things as administrator in Linux).

Step 1.4 — Make sure you are on WSL 2

Back in PowerShell:

```
wsl --set-default-version 2
wsl --set-version Ubuntu-22.04 2
wsl --set-default Ubuntu-22.04
```

Step 1.5 — Update Ubuntu

Open Ubuntu (Start menu → **Ubuntu 22.04**) and run:

```
sudo apt update && sudo apt upgrade -y
```

Type your password when asked. This may take a few minutes.

Part 2 — Linux survival kit (commands you must know)

You will type commands into the Ubuntu terminal. Here are the only ones you need for this guide.

Command	What it does
`pwd`	Show which folder you are currently in
`ls` / `ls -la`	List files in the current folder (with details)
`cd foldername`	Go *into* a folder
`cd ..`	Go *up* one folder
`cd ~`	Go to your home folder
`mkdir name`	Create a new folder
`nano file`	Open/create a text file in a simple editor
`cat file`	Print the contents of a file
`sudo <command>`	Run a command as administrator (asks for your password)
`Ctrl` + `C`	Stop the current running command

How to use the `nano` editor (you will use it a few times)

1. Run e.g. `nano .env` — the editor opens.
2. Type or paste your text. (Paste in Windows Terminal = right-click, or `Ctrl`+`Shift`+`V`.)
3. Press `Ctrl` + `O`, then `Enter` to **save**.
4. Press `Ctrl` + `X` to **exit**.

Part 3 — Enable systemd inside WSL

This lets Docker and Nginx run as proper background services (start automatically, use `systemctl`). Run inside Ubuntu:

```
sudo nano /etc/wsl.conf
```

Paste these lines, then save (`Ctrl`+O, Enter) and exit (`Ctrl`+X):

```
[boot]
systemd=true
```

Now fully restart WSL. Open **PowerShell** (normal, not Ubuntu) and run:

```
wsl --shutdown
```

Wait 10 seconds, then open Ubuntu again. Verify systemd is on:

```
systemctl is-system-running
```

It should print `running` or `degraded` — both are fine.

Part 4 — Install Docker Engine inside WSL

> Do **not** install Docker Desktop. Install Docker Engine directly in Ubuntu as shown below.

Step 4.1 — Install Docker

Copy-paste this whole block into Ubuntu and press Enter:

```
# Remove any old versions
sudo apt remove -y docker docker-engine docker.io containerd runc 2>/dev/null

# Install prerequisites
sudo apt update
sudo apt install -y ca-certificates curl gnupg

# Add Docker's official GPG key
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

# Add the Docker repository
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo $VERSION_CODENAME)
stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Install Docker Engine + Compose plugin
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-
plugin
```

Step 4.2 — Run Docker without typing `sudo` every time

```
sudo usermod -aG docker $USER
```

Then close the Ubuntu window and open it again (so the group change applies).

Step 4.3 — Start Docker and test it

```
sudo systemctl enable --now docker
docker run --rm hello-world
```

If you see **"Hello from Docker!"**, Docker works. Check versions:

```
docker --version
docker compose version
```

Part 5 — Install Nginx inside WSL

```
sudo apt update
sudo apt install -y nginx openssl
sudo systemctl enable nginx
```

Test it is installed:

```
nginx -v
```

Part 6 — Create the SSL certificate

The site runs on HTTPS, so Nginx needs a certificate. For local development we create a **self-signed** certificate at the exact paths the config expects.

```
sudo mkdir -p /etc/nginx/ssl

sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/nginx/ssl/pvkey.key \
  -out /etc/nginx/ssl/certificate.crt \
  -subj "/C=US/ST=State/L=City/O=Avnik/OU=IT/CN=staging.avnik.com"
```

Confirm the two files exist:

```
ls -la /etc/nginx/ssl
```

You should see `certificate.crt` and `pvkey.key`.

> Because it is self-signed, the browser will show a "Not secure / your connection is not private" warning the first time. That is expected on a local machine — click **Advanced → Proceed to staging.avnik.com**.

Part 7 — Add the Nginx site configuration

This is the routing file that connects each URL to the right container. Create it:

```
sudo nano /etc/nginx/sites-available/staging.avnik.com
```

Paste the **entire** configuration below, then save (`Ctrl`+O, Enter) and exit (`Ctrl`+X):

```
server {
    listen 80;
    listen [::]:80;
    server_name staging.avnik.com;
    return 301 https://$host$request_uri;
}

server {
    server_name  staging.avnik.com;
    set $csp_nonce $request_id;
    listen 443 ssl;
    listen [::]:443 ssl;

    ssl_certificate     /etc/nginx/ssl/certificate.crt;
    ssl_certificate_key /etc/nginx/ssl/pvkey.key;

    location = /myadmin { return 301 /myadmin/; }

    location /myadmin/ {
        proxy_pass http://localhost:8080/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location ~* ^/join/.*\.mjs$ {
        proxy_pass http://localhost:3002;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_hide_header Content-Type;
        add_header Content-Type "application/javascript" always;
    }

    location ~* ^/admin/.*\.mjs$ {
        proxy_pass http://localhost:8002;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_hide_header Content-Type;
        add_header Content-Type "application/javascript" always;
    }

    location ~* ^/.*\.mjs$ {
        proxy_pass http://localhost:8000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_hide_header Content-Type;
        add_header Content-Type "application/javascript" always;
    }

    # Laravel website
    location / {
        proxy_pass http://localhost:8000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

```
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # React admin
    location /admin/ {
        proxy_pass http://localhost:8002;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # Meet frontend
    location /join/ {
        proxy_pass http://localhost:3002;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location ~* \.mjs$ {
        proxy_pass http://127.0.0.1:3002;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_hide_header Content-Type;
        add_header Content-Type "application/javascript" always;
    }

    # Node admin API
    location /adminapis {
        proxy_pass http://localhost:8001;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # NEST API (meet backend)
    location /meet/api/ {
        proxy_pass http://localhost:3000;
        proxy_ssl_server_name on;
        proxy_ssl_protocols TLSv1.2 TLSv1.3;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_redirect off;
    }

    # Meet backend websocket
    location /socket.io/ {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
```

```
    }

    # Media server websocket
    location /mediasoup-socket.io/ {
        proxy_pass http://localhost:3001;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }
}
```

Enable the site and remove the default

```
sudo ln -sf /etc/nginx/sites-available/staging.avnik.com /etc/nginx/sites-enabled/
staging.avnik.com
sudo rm -f /etc/nginx/sites-enabled/default
```

Test the configuration is valid

```
sudo nginx -t
```

It should say `syntax is ok` and `test is successful`. (Do not reload yet — we start Nginx in Part 13.)

Part 8 — Point the domain (Windows hosts file)

Your browser must send `staging.avnik.com` to your own machine. WSL2 automatically forwards `127.0.0.1` from Windows into Ubuntu, so we point the domain to `127.0.0.1`.

Step 8.1 — Edit the Windows hosts file

1. Click Windows **Start**, type `Notepad`.
2. Right-click **Notepad** → **Run as administrator**.
3. In Notepad: **File** → **Open**.
4. Paste this path in the filename box and press Enter:
   `C:\Windows\System32\drivers\etc\hosts`
   (If you see no file, change the file-type dropdown to **All Files (*.*)**.)
5. Add this line at the bottom:

```
127.0.0.1 staging.avnik.com
```

1. Save with **File** → **Save** (`Ctrl`+S).

Step 8.2 — Also add it inside WSL (so tests from Ubuntu work)

```
echo "127.0.0.1 staging.avnik.com" | sudo tee -a /etc/hosts
```

Part 9 — Get the project files into WSL

> **Keep the code inside the Linux filesystem** (your home folder `~`), not under `/mnt/c/...`. Running Docker builds from the Windows drive is extremely slow and causes file-permission problems.

Step 9.1 — Create a working folder

```
mkdir -p ~/TRAINING
cd ~/TRAINING
```

Step 9.2 — Clone both repositories

Install git if needed and clone (replace the URLs with your actual Git repository URLs):

```
sudo apt install -y git

cd ~/TRAINING
git clone <URL-of-avnik-admin-repo>   avnik-admin
git clone <URL-of-avnik-meeting-repo> avnik-meeting
```

> If you use a private repo, log in when Git asks, or set up an SSH key / personal access token. Ask the team lead for the repository URLs and access.

After cloning you should have:

```
~/TRAINING/avnik-admin
~/TRAINING/avnik-meeting
```

Step 9.3 — Switch to the correct branch (if required)

```
cd ~/TRAINING/avnik-admin
git checkout dev        # or main, as the team tells you

cd ~/TRAINING/avnik-meeting
git checkout dev
```

Part 10 — Create the `.env` files

Each project needs a `.env` file (it is **not** committed to Git). Create them exactly as below.

Step 10.1 — avnik-admin/.env

```
cd ~/TRAINING/avnik-admin
nano .env
```

Paste the following, then save and exit:

```
# MySQL
MYSQL_ROOT_PASSWORD=rootpassword
MYSQL_DATABASE=avnik_db
MYSQL_USER=avnik_user
MYSQL_PASSWORD=avnik_password
MYSQL_PUBLISHED_PORT=3306

# phpMyAdmin
PHPMYADMIN_PORT=8080

# Node admin API
ADMIN_BACKEND_PORT=8001
JWT_SECRET=dev_jwt_secret_change_in_production
SKIP_DB_SYNC=true

# Laravel backend
LARAVEL_BACKEND_PORT=8000
APP_NAME=Avnik
APP_ENV=production
APP_DEBUG=true
APP_KEY=base64:aNy0v4g9SwnNpRM25/vy4hdwtO5VVkhDrQiEpF2OlI8=
LARAVEL_APP_URL=https://staging.avnik.com
FORCE_HTTPS=true
SKIP_MIGRATIONS=true

# React admin dashboard
ADMIN_FRONTEND_PORT=8003
ADMIN_FRONTEND_HTTP_PORT=8002
VITE_APP_BASE_PATH=/admin
FRONTEND_VITE_API_BASE_URL=/api
FRONTEND_VITE_IMAGE_BASE_URL=/images
FRONTEND_VITE_MEETING_FRONT_URL=http://localhost:8000
FRONTEND_VITE_MEETING_API_URL=http://localhost:8000/api

# Dev tooling
CHOKIDAR_USEPOLLING=true
NPM_CONFIG_LEGACY_PEER_DEPS=true
VITE_DEV_PROXY_TARGET=http://admin-backend:8081
```

Step 10.2 — avnik-meeting/.env

```
cd ~/TRAINING/avnik-meeting
nano .env
```

Paste the following, then save and exit:

```
# Postgres
POSTGRES_USER=meet
POSTGRES_PASSWORD=meet
POSTGRES_DB=meet

# meet-backend
BACKEND_PORT=3000
BACKEND_DATABASE_URL=postgres://meet:meet@postgres:5432/meet
BACKEND_REDIS_URL=redis://redis:6379
BACKEND_MEDIA_JWT_SECRET=1234567890
BACKEND_MEDIA_JWT_TTL_SEC=86400
BACKEND_CORS_ORIGIN='*'
BACKEND_MEETING_REDIRECT_URL='https://staging.avnik.com'
BACKEND_DATABASE_SYNC=false
BACKEND_DATABASE_LOGGING=false
BACKEND_RECORDER_SERVICE_URL=http://meet-recorder:4000
BACKEND_API_PREFIX=meet/api

# meet-mediaserver
MEDIASERVER_PORT=3001
MEDIASERVER_MEDIA_JWT_SECRET=1234567890
MEDIASERVER_LOG_LEVEL=info
MEDIASERVER_ANNOUNCED_IP=127.0.0.1,host.docker.internal
MEDIASERVER_RTC_MIN_PORT=45000
MEDIASERVER_RTC_MAX_PORT=45100
MEDIASERVER_TURN_URLS=turns:meet-staging.avnik.com:5349?transport=tcp,turn:meet-
staging.avnik.com:3478?transport=tcp,turn:meet-staging.avnik.com:3478?transport=udp
MEDIASERVER_TURN_USERNAME=avnik
MEDIASERVER_TURN_PASSWORD=avnik@123

# meet-frontend
FRONTEND_HTTP_PORT=3002
FRONTEND_VITE_MEETING_API_URL=''
FRONTEND_VITE_MEDIA_WS_URL=''
FRONTEND_VITE_MEETING_REDIRECT_URL='https://staging.avnik.com'

# meet-recorder
RECORDER_PORT=4000
RECORDER_FRONTEND_URL=http://meet-frontend:5173/join
RECORDER_MEETING_API_URL=http://meet-backend:3000
RECORDER_PUBLIC_URL=http://127.0.0.1:4000
RECORDER_MEDIA_PUBLIC_WS_URL=
RECORDER_CHUNK_INTERVAL_MS=5000
RECORDER_READY_TIMEOUT_MS=120000
RECORDER_STOP_GRACE_MS=15000
RECORDER_MAX_PARALLEL_RECORDINGS=10
```

> These values are the working local/staging defaults. If the team gives you different secrets (Stripe keys, passwords, etc.), use those instead.

Part 11 — Build & run avnik-admin

```
cd ~/TRAINING/avnik-admin
docker compose -f compose.prod.yaml up --build -d
```

What this does:

- `-f compose.prod.yaml` — use the production compose file.
- `up` — create and start the containers.
- `--build` — build the images from the code first.
- `-d` — run in the background ("detached").

The first build downloads a lot and can take **10–25 minutes**. Be patient. When it finishes, check everything is running:

```
docker compose -f compose.prod.yaml ps
```

All services should show `running` (and MySQL `healthy`). To watch the logs:

```
docker compose -f compose.prod.yaml logs -f
```

Press `Ctrl`+C to stop watching logs (this does **not** stop the containers).

Part 12 — Build & run avnik-meeting

```
cd ~/TRAINING/avnik-meeting
docker compose -f compose.prod.yaml up --build -d
```

Again wait for the build to finish, then verify:

```
docker compose -f compose.prod.yaml ps
```

> The `meet-mediaserver` uses host networking and opens UDP ports 45000–45100 for live audio/video. With Docker Engine inside WSL2 this works out of the box — no extra setup.

Part 13 — Start Nginx & open the apps

Step 13.1 — Start Nginx

```
sudo nginx -t
sudo systemctl restart nginx
sudo systemctl status nginx --no-pager
```

`status` should show **active (running)**. Press `q` to exit the status view.

Step 13.2 — Quick test from inside WSL

```
curl -k -I https://staging.avnik.com
```

`-k` ignores the self-signed cert warning. You should get an `HTTP/... 200` or a redirect.

Step 13.3 — Open in the Windows browser

What	URL
Main Laravel site	`https://staging.avnik.com/`
React admin dashboard	`https://staging.avnik.com/admin/`
Meeting frontend	`https://staging.avnik.com/join/`
phpMyAdmin (database UI)	`https://staging.avnik.com/myadmin/`

> On first visit the browser warns about the self-signed certificate. Click **Advanced → Proceed to staging.avnik.com (unsafe)**. This is safe on your own machine.

Part 14 — Daily start / stop routine

After the first setup, this is all you do each day.

Start everything

```
cd ~/TRAINING/avnik-admin   && docker compose -f compose.prod.yaml up -d
cd ~/TRAINING/avnik-meeting && docker compose -f compose.prod.yaml up -d
sudo systemctl start nginx
```

(No `--build` needed unless the code changed. Docker starts automatically with systemd enabled.)

Stop everything

```
cd ~/TRAINING/avnik-admin   && docker compose -f compose.prod.yaml down
cd ~/TRAINING/avnik-meeting && docker compose -f compose.prod.yaml down
```

Rebuild after pulling new code

```
cd ~/TRAINING/avnik-admin
git pull
docker compose -f compose.prod.yaml up --build -d
```

Part 15 — Troubleshooting

Problem	Fix
Browser: "This site can't be reached"	Nginx not running → `sudo systemctl restart nginx`. Check the hosts-file line from Part 8 exists.
Nginx `502 Bad Gateway`	The container behind that path is not up. Run `docker compose -f compose.prod.yaml ps` and check `logs`.
`bind: address already in use`	Another program uses that port. Find it: `sudo ss -ltnp | grep :8000`, then stop that program.
Docker command "permission denied"	You skipped Step 4.2, or did not reopen the terminal. Run `sudo usermod -aG docker $USER` and reopen Ubuntu.
Docker not running after reboot	`sudo systemctl enable --now docker`. Make sure systemd is enabled (Part 3).
Build is painfully slow / permission errors	The code is under `/mnt/c/`. Move it into `~/TRAINING` (Linux filesystem) as in Part 9.
Meeting audio/video won't connect	Confirm you used Docker Engine in WSL (not Docker Desktop) and that `meet-mediaserver-prod` is running.
Changed `.env` but nothing changed	Recreate containers: `docker compose -f compose.prod.yaml up -d --force-recreate`.
Need to wipe the databases and start fresh	`docker compose -f compose.prod.yaml down -v` (the `-v` deletes the data volumes).

Useful inspection commands

```
# See running containers (all projects)
docker ps

# Logs of one service
docker compose -f compose.prod.yaml logs -f laravel-backend

# Open a shell inside a container
docker compose -f compose.prod.yaml exec laravel-backend sh

# Nginx error log
sudo tail -n 50 /var/log/nginx/error.log
```

Appendix — Command cheat sheet

Goal	Command (run in Ubuntu)
Restart WSL (from PowerShell)	`wsl --shutdown`
Start Docker	`sudo systemctl start docker`
Start Nginx	`sudo systemctl start nginx`
Test Nginx config	`sudo nginx -t`
Reload Nginx after config change	`sudo systemctl reload nginx`
Bring a project up	`docker compose -f compose.prod.yaml up --build -d`
Bring a project down	`docker compose -f compose.prod.yaml down`
Status of a project	`docker compose -f compose.prod.yaml ps`
Follow logs	`docker compose -f compose.prod.yaml logs -f`
Free disk from old images	`docker system prune -a`

End of guide. Order of first-time setup: Part 1 → 15. After that, use Part 14 daily. If a step fails, check Part 15 before asking for help — include the exact command you ran and the error text.